# Informe Técnico:
# Sistema de Gestión de Guardería Central

**Materia:** Programación II – INSPT UTN

**Comisión:** 2.603 – Turno Noche

**Docente:** Miguel Silva

**Carrera:** Tecnicatura Universitaria en Informática Aplicada – UTN, INSPT

**Año:** 2026

**Proyecto:** GuarderíaCentral – Sistema de Gestión de Guardería Central

**Grupo A – Tema 09**

**Integrantes:**

Franco Buyatti

Daniela Forclaz

Héctor Machaca

María Eugenia Barca

# ÍNDICE

**2. Link al repositorio de GitHub..... 2**
**3. Descripción general..... 2**
**4. Arquitectura y desarrollo..... 3**
**5. Casos de uso y diagramas..... 11**
**6. Base de datos..... 13**
**7. Manual de usuario..... 16**
**8. Decisiones de implementación..... 27**
**9. Uso de los principios de POO..... 29**
**10. Aplicación de los principios S.O.L.I.D..... 29**
**11. Aclaraciones relevantes..... 30**

## 2. Link al repositorio de GitHub

**Repositorio del backend:** https://github.com/mariabarca-dev/Guarderia_Backend

**Repositorio del frontend:** https://github.com/mariabarca-dev/Guarderia_Frontend

## 3. Descripción general

### Tema elegido

El tema es la gestión de una guardería de rodados, la **Guardería Central**, que guarda motorhomes, casas rodantes de arrastre, caravanas y trailers. La guardería se divide en zonas, cada una destinada a un solo tipo de rodado, y cada zona tiene cocheras numeradas de medidas fijas. Los socios compran cocheras y las usan para guardar sus rodados, y los empleados se ocupan de los rodados de cada zona.

### Objetivos del sistema

Registrar zonas, cocheras y rodados, controlando que cada rodado entre en las medidas y en el tipo de la zona donde se guarda.

Registrar quién es el dueño de cada cochera (compra) y qué rodado la ocupa en cada momento, evitando que una cochera tenga dos dueños o dos rodados. Administrar las cuentas de socios, empleados y administradores, y asignar a cada empleado los sectores (zonas) y los rodados que tiene a cargo.

Conservar el historial: nada se borra, las bajas son lógicas y se pueden consultar. Dar a cada tipo de usuario solo las pantallas y operaciones que le corresponden.

### Actores y sus roles

El sistema tiene cuatro actores, cada uno con funciones distintas (el detalle de permisos está en la tabla de roles de la sección 4):

**Socio:** consulta sus propios datos, rodados, cocheras y compras.

**Empleado:** consulta su legajo, los sectores que tiene asignados y los rodados a su cargo.

**Administrador:** gestiona la guardería (zonas, cocheras, rodados, compras, sectores y ocupaciones) y las cuentas de socios, empleados y administradores. **SYSADMIN:** es el administrador del sistema. Ve todas las cuentas y gestiona las de socios, empleados y administradores. Se crea automáticamente al iniciar el backend.

## 4. Arquitectura y desarrollo

El sistema son **dos aplicaciones independientes** que se comunican por HTTP: un **backend** con la lógica de negocio y la base de datos, y un **frontend** web que solo muestra pantallas. Cada una está organizada en capas, para que la lógica de negocio, el acceso a datos y la interfaz queden desacoplados.

**Flujo general:** Navegador → Frontend (Spring MVC + JSP, puerto 8081) → Backend (API REST con JSON, puerto 8080) → MySQL.

### Backend (API REST)

**Capa Model (Entidades):** define los objetos del dominio con JPA: Usuario (abstracta) con sus subtipos Socio , Empleado y Administrador , más Zona , Garage (cochera), Vehiculo (rodado), PropiedadGarage (compra de una cochera) y AsignacionVehiculoGarage (ocupación). AsignacionEmpleadoZona representa el sector de un empleado en una zona.

**Capa Repository (Acceso a Datos):** interfaces de Spring Data JPA. Junto a cada repositorio viven sus objetos de transferencia: Request (alta), Update (modificación) y Response (salida), con los métodos que convierten entre DTO y entidad. **Capa Service (Lógica de Negocio):** una interfaz y su implementación por entidad. Ahí se aplican las reglas: unicidad de DNI, matrícula y número de cochera, capacidad de las zonas, exclusividad de la ocupación de una cochera y bajas con dependencias. **Capa Controller (REST):** controladores bajo /api/... que reciben la solicitud, validan los datos y delegan en el service.

**Capa Exceptions:** excepciones de negocio propias y un GlobalExceptionHandler que las convierte en respuestas JSON con el código HTTP correspondiente (400, 403, 404, 409).

**Capa Security:** autenticación por sesión HTTP y autorización por rol, concentrada en una sola clase ( SecurityConfig ).

### Frontend (Spring MVC + JSP)

**Controllers:** uno por entidad. Arman el modelo y devuelven siempre la vista layout , que incluye el menú del rol y la pantalla pedida.

**Services:** uno por entidad. Llaman al backend mediante BackendClientService , el único punto de contacto HTTP.

**Vistas JSP:** una por entidad y un menú por rol. Por consigna **no se usa JavaScript**: todo se resuelve con formularios, enlaces y JSTL.

**Seguridad:** el login se valida contra el backend; el frontend protege las rutas por prefijo ( /socio , /empleado , /administrador , /sysadmin ) y usa protección CSRF. **Sin DTO en el front:** los datos viajan como Map / List<Map> , que es lo que devuelve el backend.

**Modelo de datos**

La herencia es **JOINED**: la tabla usuarios guarda los datos comunes y cada subtipo tiene su propia tabla ( socios , empleados , administradores ). El SYSADMIN es un Administrador con rol SYSADMIN .

**Roles**

| Rol | Alcance |
|---|---|
| SOCIO | Consulta lo propio: perfil, rodados, cocheras compradas y cocheras ocupadas |

**Informe Técnico: Sistema de Gestión de Guardería Central**

| Rol | Alcance |
|---|---|
| EMPLEADO | Consulta lo propio: legajo, sectores y rodados a su cargo |
| ADMINISTRADOR | Gestiona zonas, cocheras, rodados, compras, sectores y ocupaciones, y las cuentas de socios, empleados y administradores |
| SYSADMIN | Ve todas las cuentas y gestiona las de socios, empleados y administradores |

**Estructura del proyecto**

Cada aplicación es un proyecto Maven independiente, con su propio repositorio.

**Backend** ( Guarderia_Backend , paquete base com.guarderiaCentral.guarderia_Backend ):

Guarderia_Backend/
├── entorno/config.properties (credenciales, no se sube a Git) ├── src/main/java/.../guarderia_Backend/
│ ├── config/ ConstantesNegocio, SysAdminInitializer, AppEnvironmentConfig

---

│ ├── modelos/ entidades JPA y enums (Rol, TipoVehiculo) │ ├── repositories/ un subpaquete por entidad: repositorio + Request/Update/Response
│ ├── services/ interfaces de negocio
│ │ └── impl/ implementaciones con las reglas de negocio │ ├── restcontrollers/ controladores REST (/api/...)
│ ├── security/ SecurityConfig, AuthController, AuthService │ └── exceptions/ excepciones de negocio y GlobalExceptionHandler ├── src/main/resources/application.properties
└── pom.xml

**Frontend** ( Guarderia_Frontend , paquete base com.GuarderiaCentral.Guarderia_Frontend ):

Guarderia_Frontend/
├── src/main/java/.../Guarderia_Frontend/
│ ├── controllers/ un controller por entidad (devuelven siempre la vista "layout")
│ ├── services/ un service por entidad (llaman al backend) │ ├── client/ BackendClientService y BackendException │ ├── security/ autenticación contra el backend, redirección por rol, logout
│ ├── config/ SecurityConfig, RestTemplateConfig, WebConfig │ └── advice/ datos globales para las vistas y manejo de errores
├── src/main/webapp/WEB-INF/jsp/ layout, una vista por entidad y menu/<rol>.jsp
├── src/main/resources/static/css/estilos.css
└── pom.xml

**Clases principales y sus responsabilidades**

| Clase | Aplicación | Responsabilidad |
|---|---|---|
| Usuario (abstracta), Socio , Empleado , Administrador | Backend | Entidades de las cuentas, con herencia JOINED. Usuario guarda lo común y cada subtipo agrega lo propio. |
| Zona , Garage , Vehiculo , PropiedadGarage , AsignacionVehiculoGarage , AsignacionEmpleadoZona | Backend | Entidades del negocio: zonas, cocheras, rodados, compras, ocupaciones y sectores de los empleados. |

| | | |
|---|---|---|
| XxxRepository (por entidad) | Backend | Acceso a datos con Spring Data JPA. Junto a él viven Request , Update y Response , y los métodos que convierten entre DTO y entidad. |
| XxxService y XxxServiceImpl (por entidad) | Backend | Reglas de negocio: unicidad, capacidad de zonas, exclusividad de cocheras, bajas con dependencias y validación de fechas. |
| XxxRestController (por entidad) | Backend | Recibe la petición HTTP, valida con @Valid y delega en el service. |

| Clase | Aplicación | Responsabilidad |
|---|---|---|
| SecurityConfig | Backend | Autorización por rol, concentrada en un solo lugar. |
| AuthController , AuthService | Backend | Login y logout con sesión HTTP. Verifica la clave con BCrypt. |
| GlobalExceptionHandler , BusinessException | Backend | Traducen las excepciones de negocio a JSON con el código HTTP que corresponde. |
| SysAdminInitializer | Backend | Crea el SYSADMIN inicial al arrancar, si todavía no existe. |
| ConstantesNegocio | Backend | Reglas compartidas: fecha de fundación y límite de fechas hacia adelante. |
| BackendClientService | Frontend | Único punto de contacto HTTP con el backend. Maneja la cookie de sesión y convierte los errores en BackendException . |

| | | |
|---|---|---|
| XxxController (por entidad) | Frontend | Arma el modelo de la pantalla y devuelve la vista layout . |
| XxxService (por entidad) | Frontend | Llama al backend con BackendClientService y devuelve Map / List<Map> . |
| BackendAuthenticationProvider , RedireccionPorRolHandler | Frontend | Validan el login contra el backend y redirigen a /<rol>/dash . |
| BackendExceptionAdvice , ModeloGlobalAdvice | Frontend | Muestran los errores del backend y cargan en todas las vistas el rol y el usuario. |

**Ejemplos de código relevantes**

**Controlador REST (backend):** recibe el JSON, lo valida y delega en el service. No tiene lógica de negocio.

```
@PostMapping
public ResponseEntity<ZonaResponse> registrarZona(@Valid @RequestBody ZonaRequest request) {
    log.info("REST Request para registrar nueva Zona con letra: {}", request.getLetra());
    ZonaResponse nuevaZona = zonaService.registrarZona(request); return new ResponseEntity<>(nuevaZona, HttpStatus.CREATED);
}
```

**Regla de negocio en el service (backend):** una cochera no puede tener dos dueños, y la fecha de compra se valida antes de guardar.

```
if (yaVendido) {
    log.warn("Intento fallido de compra: El garage N° {} ya tiene un socio propietario asignado.", garage.getNumeroGarage());
    throw new GarageYaVendidoException("Error: El garaje N° " + garage.getNumeroGarage() + " ya tiene un socio propietario asignado."); }

validarRangoFechaCompra(request.getFechaCompraGarage());
validarFechaCompraPosteriorAlIngreso(request.getFechaCompraGarage(),socio);
```

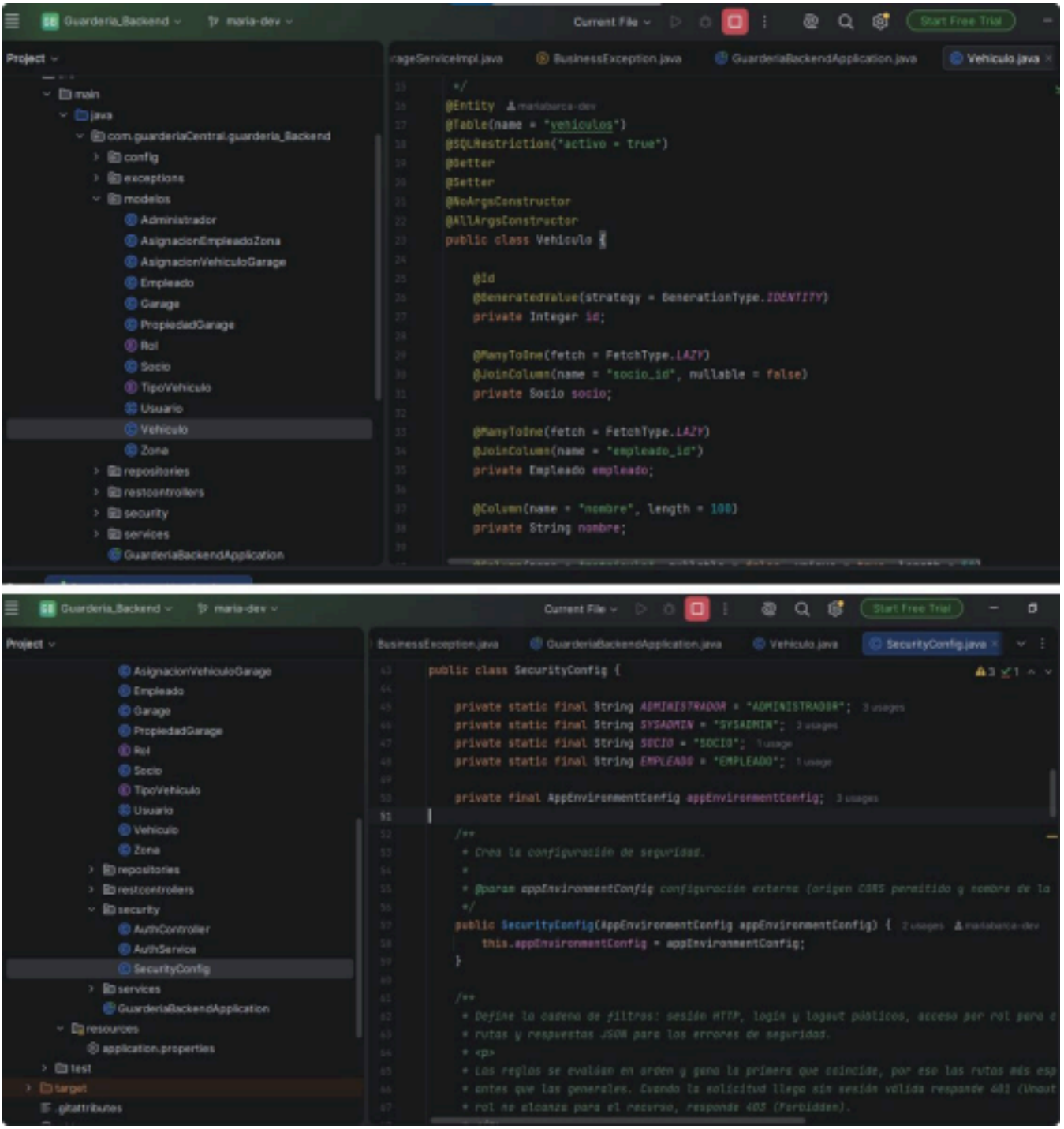
**Autorización centralizada (backend):** la primera regla que coincide es la que se aplica.

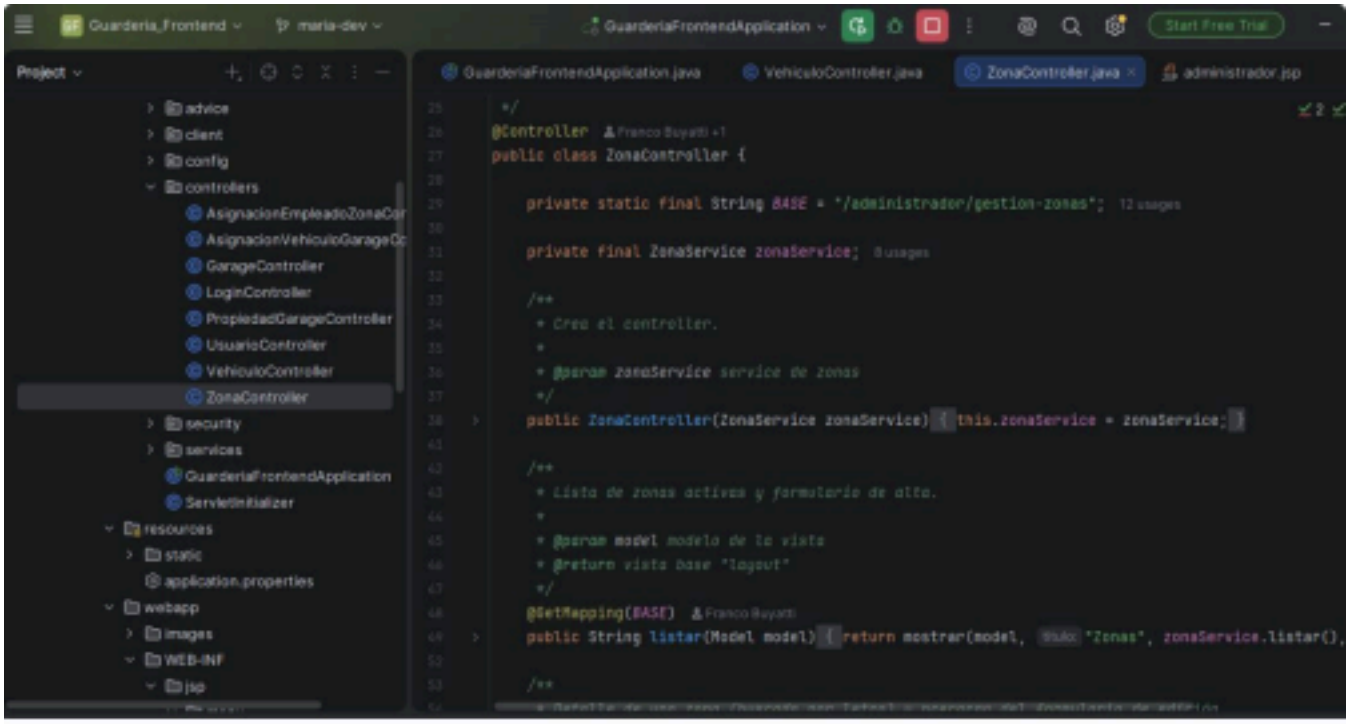
```
.requestMatchers("/api/auth/**").permitAll()
.requestMatchers("/api/usuarios/me").authenticated()
.requestMatchers("/api/usuarios/me/socio/**").hasRole(SOCIO)
.requestMatchers("/api/usuarios/me/empleado/**").hasRole(EMPLEADO)
.requestMatchers("/api/zonas/**", "/api/garages/**",
"/api/vehiculos/**").hasRole(ADMINISTRADOR)
.anyRequest().authenticated()
```

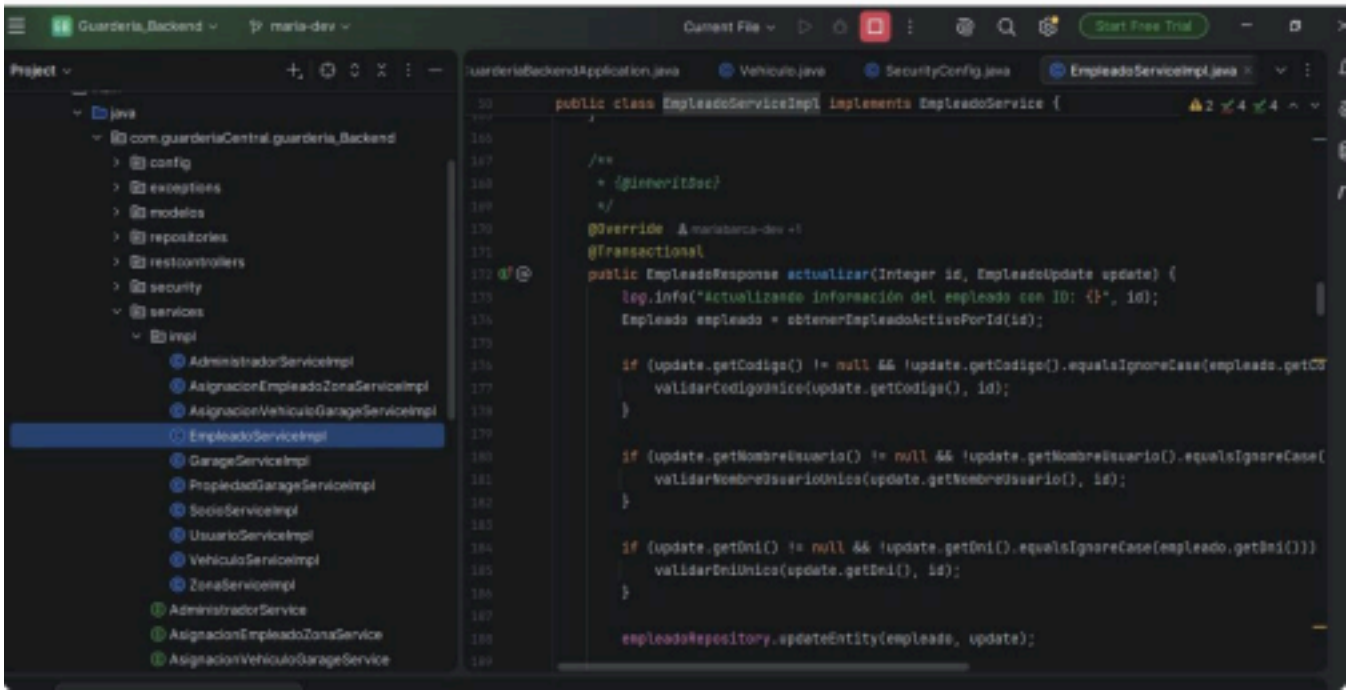
**Controller del frontend:** carga el modelo con las banderas que decide la vista y devuelve siempre layout .

```
private String mostrar(Model model, String titulo, List<Map<String, Object>> lista,
Map<String, Object> registro, boolean esBajas) {
        model.addAttribute("vista", "zona");
        model.addAttribute("titulo", titulo);
        model.addAttribute("lista", lista);
        model.addAttribute("registro", registro);
        model.addAttribute("esBajas", esBajas);
        model.addAttribute("puedeCrear", !esBajas);
        model.addAttribute("puedeEditar", !esBajas);
        model.addAttribute("puedeEliminar", !esBajas);
        return "layout";
}
```

**Capturas del código (entidad Vehiculo , autorización en SecurityConfig del back, ZonaController del front y actualizar de EmpleadoServiceImpl ):**


Guarderia_Backend
maria-dev
Current File
Start Free Trial
Project
Vehiculo.java
@Entity
@Table(name = "vehiculos")
@SQLRestriction("activo = true")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Vehiculo {
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Integer id;
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "socio_id", nullable = false)
private Socio socio;
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "empleado_id")
private Empleado empleado;
@Column(name = "nombre", length = 100)
private String nombre;
SecurityConfig.java
public class SecurityConfig {
private static final String ADMINISTRADOR = "ADMINISTRADOR";
private static final String SYSADMIN = "SYSADMIN";
private static final String SOCIO = "SOCIO";
private static final String EMPLEADO = "EMPLEADO";
private final AppEnvironmentConfig appEnvironmentConfig;
public SecurityConfig(AppEnvironmentConfig appEnvironmentConfig) {
this.appEnvironmentConfig = appEnvironmentConfig;
}


Guarderia_Frontend
maria-dev
GuarderiaFrontendApplication
Start Free Trial
Project
GuarderiaFrontendApplication.java
VehiculoController.java
ZonaController.java
administrador.jsp
advice
client
config
controllers
AsignacionEmpleadoZonaCon
AsignacionVehiculoGarageCo
GarageController
LoginController
PropiedadGarageController
UsuarioController
VehiculoController
ZonaController
security
services
GuarderiaFrontendApplication
ServletInitializer
resources
static
application.properties
webapp
images
WEB-INF
jsp
@Controller
public class ZonaController {
private static final String BASE = "/administrador/gestion-zonas";
private final ZonaService zonaService;
* Crea el controller.
* @param zonaService service de zonas
public ZonaController(ZonaService zonaService) { this.zonaService = zonaService; }
* Lista de zonas activas y formulario de alta.
* @param model modelo de la vista
* @return vista base "layout"
@GetMapping(BASE)
public String listar(Model model) { return mostrar(model, "Zonas", zonaService.listar(),


Guarderia_Backend
maria-dev
Current File
Start Free Trial
Project
GuarderiaBackendApplication.java
Vehiculo.java
SecurityConfig.java
EmpleadoServiceImpl.java
java
com.guarderiaCentral.guarderia_Backend
config
exceptions
modelos
repositories
restcontrollers
security
services
impl
AdministradorServiceImpl
AsignacionEmpleadoZonaServiceImpl
AsignacionVehiculoGarageServiceImpl
EmpleadoServiceImpl
GarageServiceImpl
PropiedadGarageServiceImpl
SocioServiceImpl
UsuarioServiceImpl
VehiculoServiceImpl
ZonaServiceImpl
AdministradorService
AsignacionEmpleadoZonaService
AsignacionVehiculoGarageService
public class EmpleadoServiceImpl implements EmpleadoService {
* {@inheritDoc}
@Override
@Transactional
public EmpleadosResponse actualizar(Integer id, EmpleadoUpdate update) {
log.info("Actualizando información del empleado con ID: {}", id);
Empleado empleado = obtenerEmpleadoActivoPorId(id);
if (update.getCodigo() != null && !update.getCodigo().equalsIgnoreCase(empleado.getCo
validarCodigoUnico(update.getCodigo(), id);
if (update.getNombreUsuario() != null && !update.getNombreUsuario().equalsIgnoreCase(
validarNombreUsuarioUnico(update.getNombreUsuario(), id);
if (update.getDni() != null && !update.getDni().equalsIgnoreCase(empleado.getDni()))
validarDniUnico(update.getDni(), id);
empleadosRepository.updateEntity(empleado, update);

## 5. Casos de uso y diagramas

El diagrama de casos de uso con los cuatro actores está en la carpeta documentación del repositorio del backend. Ahí están también los diagramas de clases (UML) del sistema. Cada caso de uso se describe a continuación.

| Código | Actor | Caso de uso | Descripción breve |
|---|---|---|---|
| CU-01 | Todos | Iniciar sesión | El usuario ingresa su usuario y clave. El sistema los valida contra el backend y lo lleva al inicio de su rol. |
| CU-02 | Todos | Ver mis datos de cuenta | El panel de inicio muestra usuario, nombre, DNI, rol y estado de la cuenta. |
| CU-03 | Todos | Cerrar sesión | Termina la sesión y vuelve a la pantalla de ingreso. La sesión también vence a los 30 minutos sin uso. |
| CU-04 | Socio | Consultar mi perfil | Muestra sus datos personales y su fecha de ingreso, en solo lectura. |
| CU-05 | Socio | Consultar mis rodados | Lista sus rodados y permite ver el detalle de cada uno. |
| CU-06 | Socio | Consultar mis cocheras | Muestra las cocheras donde tiene rodados guardados. |
| CU-07 | Socio | Consultar mis compras | Lista las cocheras que compró, con la fecha de compra. |
| CU-08 | Empleado | Consultar mi legajo | Muestra su código, especialidad y datos personales. |
| CU-09 | Empleado | Consultar mis sectores | Lista las zonas que tiene asignadas y cuántos rodados tiene a cargo en cada una. |
| CU-10 | Empleado | Consultar mis encargos | Lista los rodados a su cargo, con la letra de zona y el número de |

| | | | |
|---|---|---|---|
| | | | cochera. |
| CU-11 | Administrador | Gestionar zonas | Alta, edición y baja de zonas. Permite buscar una zona por su letra y ver las dadas de baja. |
| CU-12 | Administrador | Gestionar cocheras | Alta, edición y baja de cocheras, con su lectura de luz y servicio de mantenimiento. Permite ver las dadas de baja. |

| Código | Actor | Caso de uso | Descripción breve |
|---|---|---|---|
| CU-13 | Administrador | Gestionar rodados | Alta, edición y baja de rodados. Permite buscar por matrícula, por tipo de rodado o por DNI del socio. |
| CU-14 | Administrador | Registrar compras de cocheras | Registra qué socio compró una cochera y en qué fecha. Permite consultar las compras de un socio por su DNI. |
| CU-15 | Administrador | Registrar ocupaciones | Asigna un rodado a una cochera. Permite consultar por matrícula y ver el historial de ocupaciones. |
| CU-16 | Administrador | Asignar sectores a empleados | Asigna un empleado a una zona y define cuántos rodados tiene a cargo. Permite buscar por código de empleado y ver el historial. |
| CU-17 | Administrador | Gestionar cuentas de socios | Alta, edición y baja de socios. Permite buscar por documento y ver los rodados y las cocheras de cada socio. |
| CU-18 | Administrador | Gestionar cuentas de empleados | Alta, edición y baja de empleados. Permite encargarle o quitarle rodados y ver sus sectores y encargos. |

| | | | |
|---|---|---|---|
| CU-19 | Administrador | Gestionar cuentas de administradores | Alta, edición y baja de cuentas de administrador. No muestra las cuentas SYSADMIN. |
| CU-20 | SYSADMIN | Ver todas las cuentas | Lista todas las cuentas del sistema, activas y dadas de baja, con el detalle de cada una (solo lectura). |
| CU-21 | SYSADMIN | Gestionar cuentas | Alta, edición y baja de cuentas de socios, empleados y administradores. |

En todos los casos de gestión, la baja es lógica y el sistema la rechaza si hay registros activos que dependen del que se quiere dar de baja.

## 6. Base de datos

El motor es MySQL, en la base guarderia_db . El diagrama entidad-relación está en la carpeta documentación del repositorio del backend. Las tablas las genera Hibernate al iniciar el backend ( ddl-auto=update ), a partir de las entidades.

**Tablas**

| Tabla | Campos principales | Descripción |
|---|---|---|
| usuarios | id, nombre, apellido, dni (único, opcional), direccion, telefono, nombre_usuario (único), clave (BCrypt), rol, activo | Datos comunes de todas las cuentas. |
| socios | usuario_id (PK y FK a usuarios ), fecha_ingreso | Datos propios del socio. |
| empleados | usuario_id (PK y FK a usuarios ), codigo (único), especialidad | Datos propios del empleado. |

| | | |
|---|---|---|
| administradores | usuario_id (PK y FK a usuarios ) | Cuentas de administrador. El SYSADMIN es una fila con rol SYSADMIN. |
| zonas | id, letra (única), tipo_vehiculo, capacidad_vehiculos, ancho_garage, largo_garage, activo | Zonas de la guardería. Cada una admite un solo tipo de rodado. |
| garages | id, numero_garage (único), lectura_luz, servicio_mantenimiento, zona_id (FK), activo | Cocheras numeradas. |
| vehiculos | id, socio_id (FK), empleado_id (FK, opcional), nombre, matricula (única), tipo, profundidad, ancho, activo | Rodados de los socios. El empleado a cargo es opcional. |
| propiedades_garage | id, socio_id (FK), garage_id (FK), fecha_compra_garage, activo | Compra de una cochera por un socio. |

| Tabla | Campos principales | Descripción |
|---|---|---|
| asignaciones_vehiculo_garage | id, vehiculo_id (FK), garage_id (FK), fecha_asignacion_garage, activo | Ocupación de una cochera por un rodado. |
| asignaciones_empleado_zona | id, empleado_id (FK), zona_id (FK), cant_vehiculos_a_cargo, activo | Sector de un empleado: la zona que cubre y cuántos rodados tiene a cargo en ella. |

**Relaciones**

| Relación | Tipo | Detalle |
|---|---|---|
| usuarios → socios / empleados / administradores | Herencia (JOINED) | Cada cuenta tiene una fila en usuarios y una en la tabla de su subtipo, con el mismo id. |
| zonas → garages | 1 a N | Una zona tiene muchas cocheras y cada cochera pertenece a una zona. |
| socios → vehiculos | 1 a N | Un socio tiene muchos rodados y cada rodado pertenece a un socio. |
| empleados → vehiculos | 1 a N (opcional) | Un empleado puede tener a cargo muchos rodados y un rodado tiene como mucho un empleado a cargo. |
| socios → propiedades_garage ← garages | N a M (con datos) | Un socio puede comprar muchas cocheras. Cada cochera tiene un solo dueño activo, y la compra guarda la fecha. |
| vehiculos → asignaciones_vehiculo_garage ← garages | N a M (con datos) | Un rodado ocupa una cochera por vez y una cochera tiene un solo rodado activo. La asignación guarda la fecha. |

| Relación | Tipo | Detalle |
|---|---|---|
| empleados → asignaciones_empleado_zona ← zonas | N a M (con datos) | Un empleado puede cubrir varias zonas y una zona puede tener varios empleados. La asignación guarda cuántos rodados tiene a cargo. |

**Reglas de integridad que aplica el sistema**

Son únicos el DNI, el nombre de usuario, el código de empleado, la matrícula, la letra de zona y el número de cochera.

Todas las tablas de negocio tienen el campo activo : la baja es lógica y no se pierde el historial.

Un rodado solo se asigna a una cochera de una zona de su mismo tipo y de medidas suficientes.

Las fechas de ingreso, compra y ocupación no pueden ser anteriores a la fundación de la guardería (01/01/1926) ni superar en más de un año a la fecha actual. La compra de una cochera tampoco puede ser anterior al ingreso del socio.

## 7. Manual de usuario

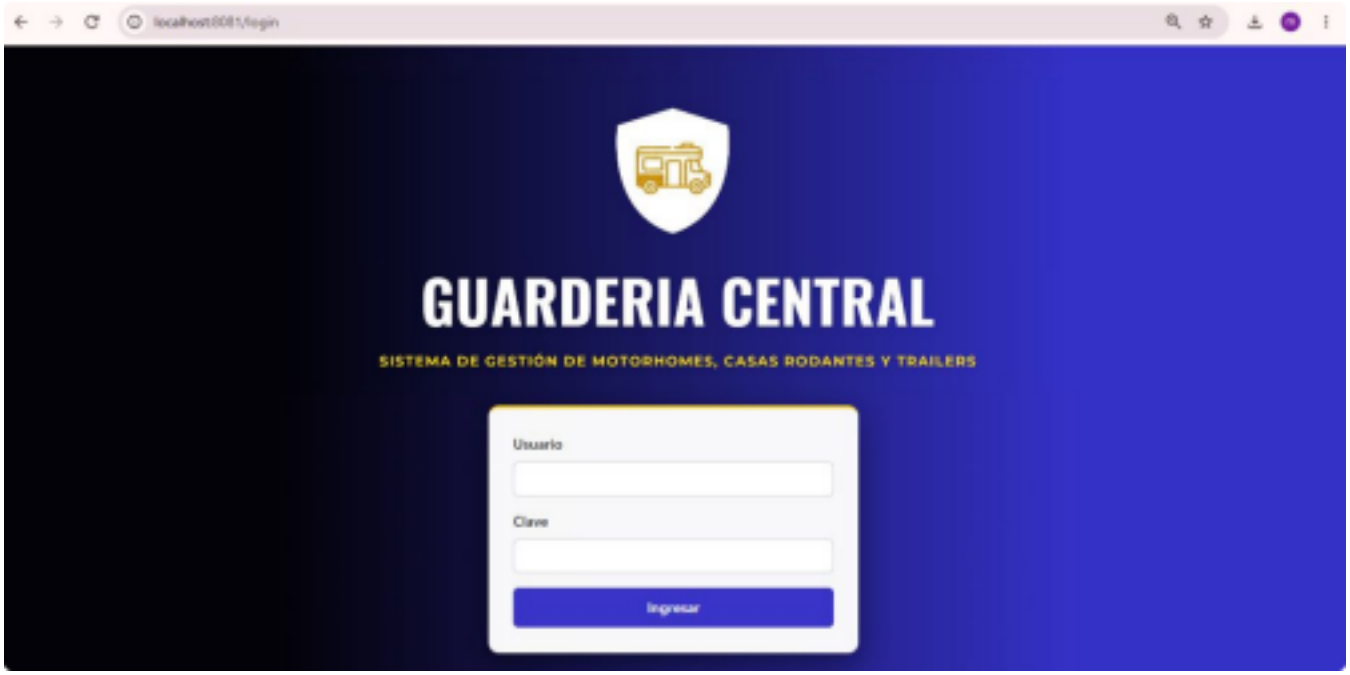
Cómo acceder al sistema (todos los actores)

1. Verificá que estén en ejecución MySQL, el backend (puerto 8080) y el frontend (puerto 8081), en ese orden.
2. Abrí en el navegador http://localhost:8081/login . Desde otra PC o un celular, usá la IP de la PC donde corre el frontend: http://<IP>:8081/login .
3. Ingresá el **Usuario** y la **Clave** y presioná **Ingresar**.
4. El sistema te lleva al inicio de tu rol. Ahí ves tus datos de cuenta (usuario, nombre, DNI, rol y estado), y arriba el menú con las opciones que te corresponden. 5. Para salir, usá el botón **Cerrar sesión** (o **Salir**) del menú. La sesión también se cierra sola a los 30 minutos sin actividad.

Si hay un error (por ejemplo, un dato inválido o un registro repetido), la pantalla muestra un aviso rojo arriba con el motivo, y no se guarda nada.

En el menú, la opción de la pantalla donde estás parado se muestra con el texto dorado.

**Pantalla de ingreso (login):**

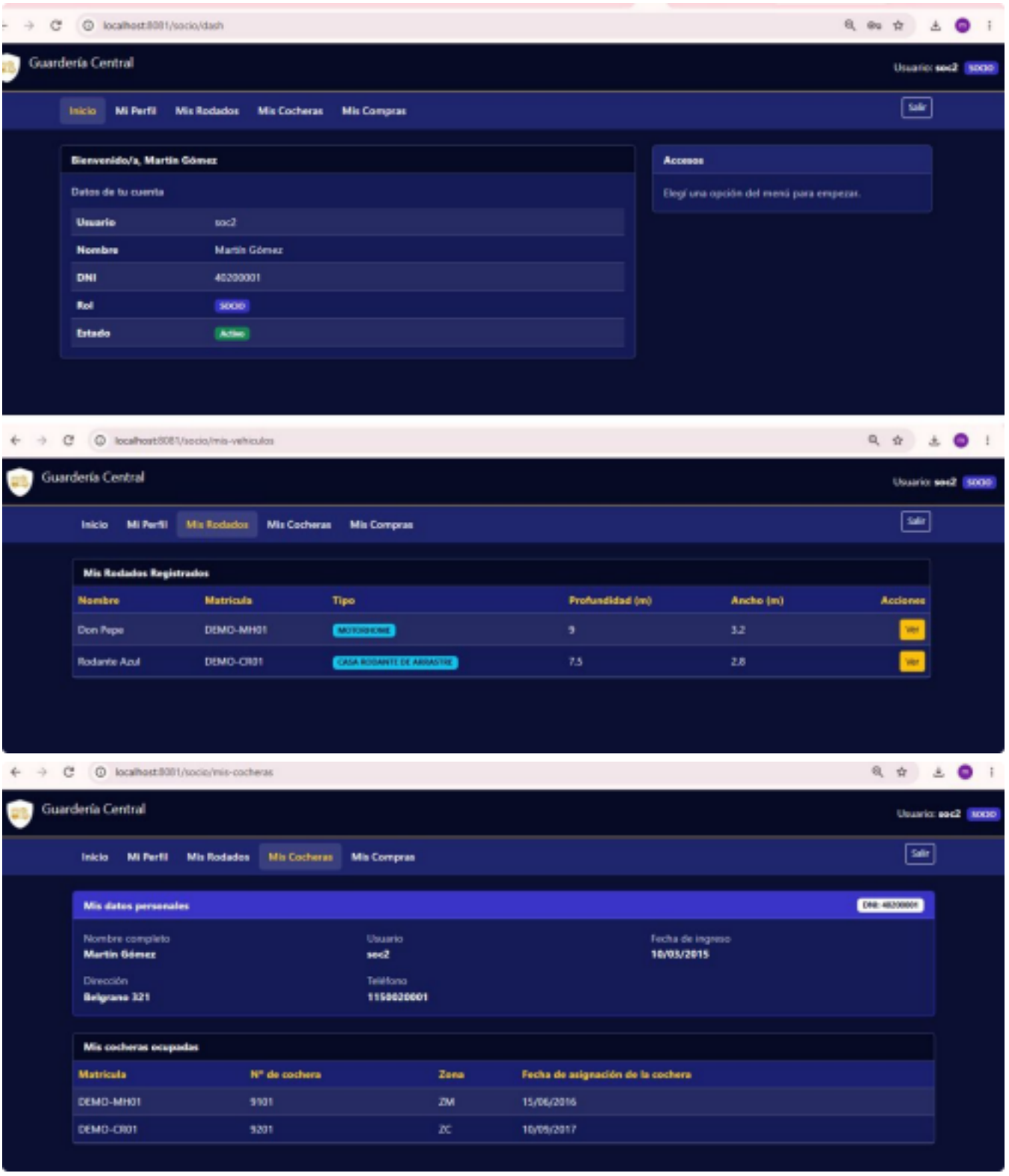



**Manual del Socio**

**Menú:** Inicio, Mi Perfil, Mis Rodados, Mis Cocheras, Mis Compras. El socio solo consulta: no puede crear, editar ni dar de baja nada.

**Mi Perfil:** muestra tus datos personales (nombre, usuario, DNI) y tu fecha de ingreso. **Mis Rodados:** tabla con tus rodados (nombre, matrícula, tipo, profundidad y ancho). El botón **Ver** de cada fila abre el detalle.

**Mis Cocheras:** muestra las cocheras donde tenés rodados guardados. **Mis Compras:** lista las cocheras que compraste, con la fecha de compra. **Capturas del Socio:**


localhost:8081/socio/dash
Guardería Central
Usuario: soc2 SOCIO
Inicio
Mi Perfil
Mis Rodados
Mis Cocheras
Mis Compras
Salir
Bienvenido/a, Martín Gómez
Datos de tu cuenta
Usuario soc2
Nombre Martín Gómez
DNI 40200001
Rol SOCIO
Estado Activo
Accesos
Elegí una opción del menú para empezar.
localhost:8081/socio/mis-vehiculos
Guardería Central
Usuario: soc2 SOCIO
Inicio
Mi Perfil
Mis Rodados
Mis Cocheras
Mis Compras
Salir
Mis Rodados Registrados
Nombre
Matrícula
Tipo
Profundidad (m)
Ancho (m)
Acciones
Don Pepe
DEMO-MH01
MOTORHOME
9
3.2
Ver
Rodante Azul
DEMO-CR01
CASA RODANTE DE ARRASTRE
7.5
2.8
Ver
localhost:8081/socio/mis-cocheras
Guardería Central
Usuario: soc2 SOCIO
Inicio
Mi Perfil
Mis Rodados
Mis Cocheras
Mis Compras
Salir
Mis datos personales
DNI: 40200001
Nombre completo
Martín Gómez
Usuario
soc2
Fecha de ingreso
10/05/2015
Dirección
Belgrano 321
Teléfono
1150020001
Mis cocheras ocupadas
Matrícula
Nº de cochera
Zona
Fecha de asignación de la cochera
DEMO-MH01
9101
ZM
15/06/2016
DEMO-CR01
9201
ZC
10/09/2017

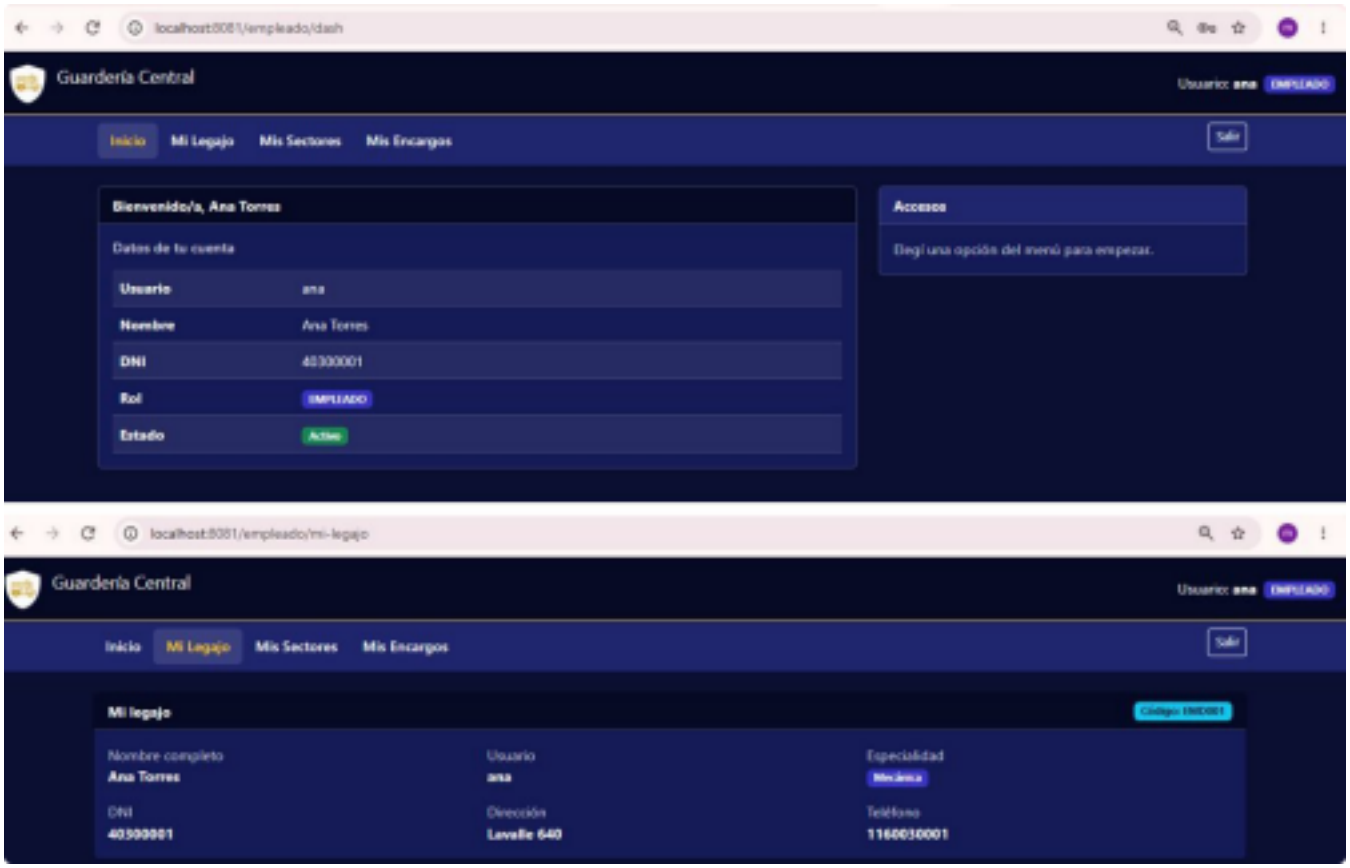
**Manual delEmpleado**

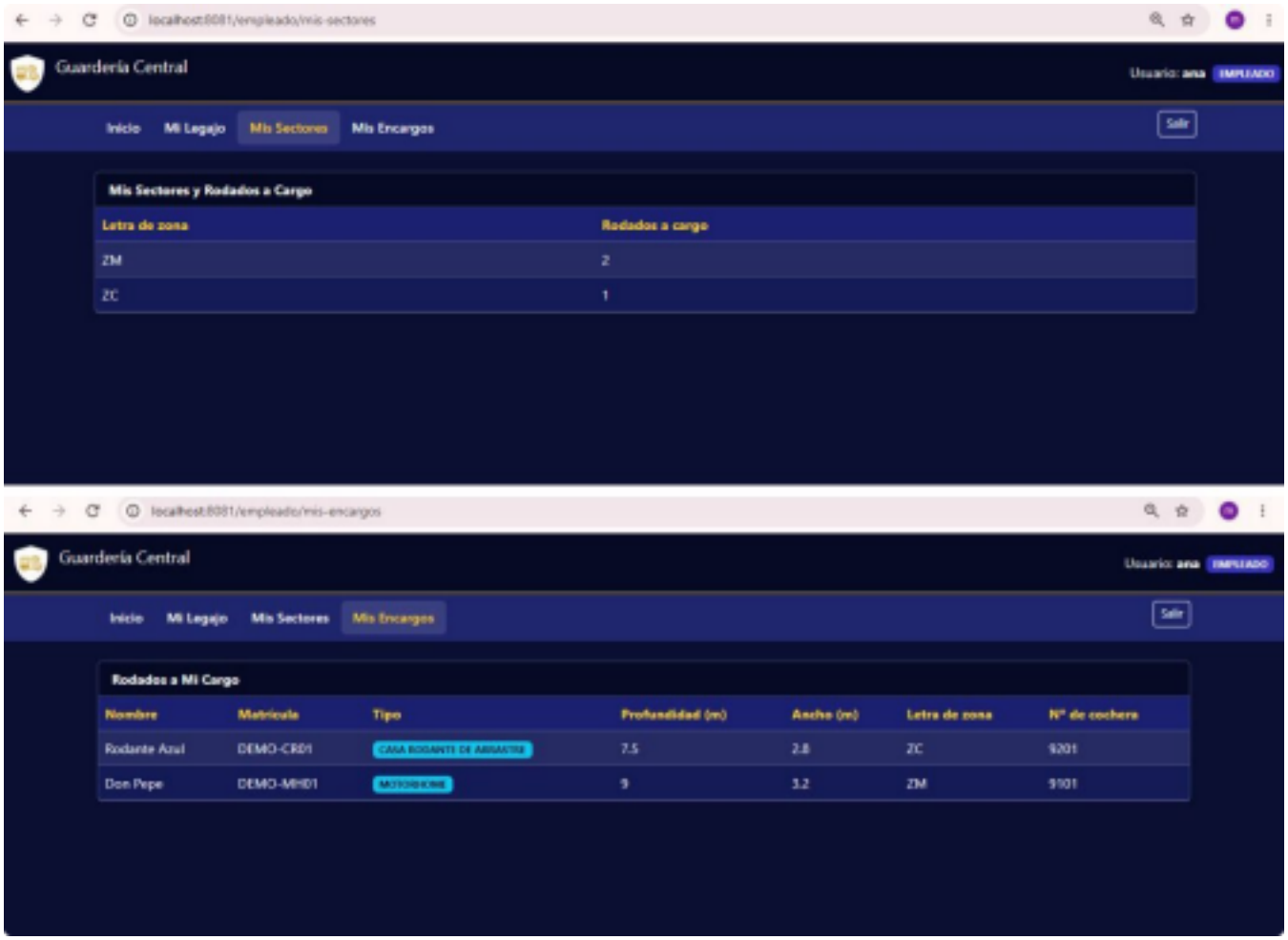
**Menú:** Inicio, Mi Legajo, Mis Sectores, Mis Encargos. El empleado solo consulta.

**Mi Legajo:** muestra tu código de empleado, tu especialidad y tus datos personales. **Mis Sectores:** lista las zonas que tenés asignadas y cuántos rodados tenés a cargo en cada una.

**Mis Encargos:** tabla con los rodados a tu cargo (nombre, matrícula, tipo, medidas), con la letra de zona y el número de cochera donde está cada uno.

**Capturas del Empleado:**






**Manual del Administrador**

**Menú:** Inicio, Zonas, Cocheras, Rodados, Compras, Sectores, Ocupaciones, Socios, Empleados, Administradores.

Las pantallas de gestión funcionan todas igual:

1. **Consultas:** en la parte de arriba, botones para ver el **Listado**, ver los registros **activos y dados de baja**, y buscar (por ejemplo, un rodado por matrícula, por tipo o por DNI del socio).

2. **Listado:** la tabla muestra los registros. Cada fila tiene **Editar** (o **Ver**) y **Dar de baja**. 3. **Crear:** el formulario de abajo ("Nuevo registro") se completa y se presiona **Registrar**. 4. **Editar:** con **Editar** el formulario se carga con los datos. Se cambian los que haga falta y se presiona **Guardar cambios**, o **Cancelar** para salir sin guardar.

5. **Dar de baja:** el registro deja de verse en el listado, pero se conserva y aparece en la vista de bajas. El sistema no permite la baja si hay registros activos que dependen de él (por ejemplo, una zona con cocheras).

**Pantallas y particularidades:**

**Zonas:** letra, tipo de rodado, capacidad y medidas de cada cochera. Se puede buscar una zona por su letra.

**Cocheras:** número, zona, lectura del contador de luz y si tiene servicio de mantenimiento (Sí o No).

**Rodados:** DNI del socio, nombre o descripción, matrícula, tipo de rodado, profundidad y ancho.

**Compras:** DNI del socio, número de cochera y fecha de compra. Se pueden consultar las compras de un socio por su DNI.

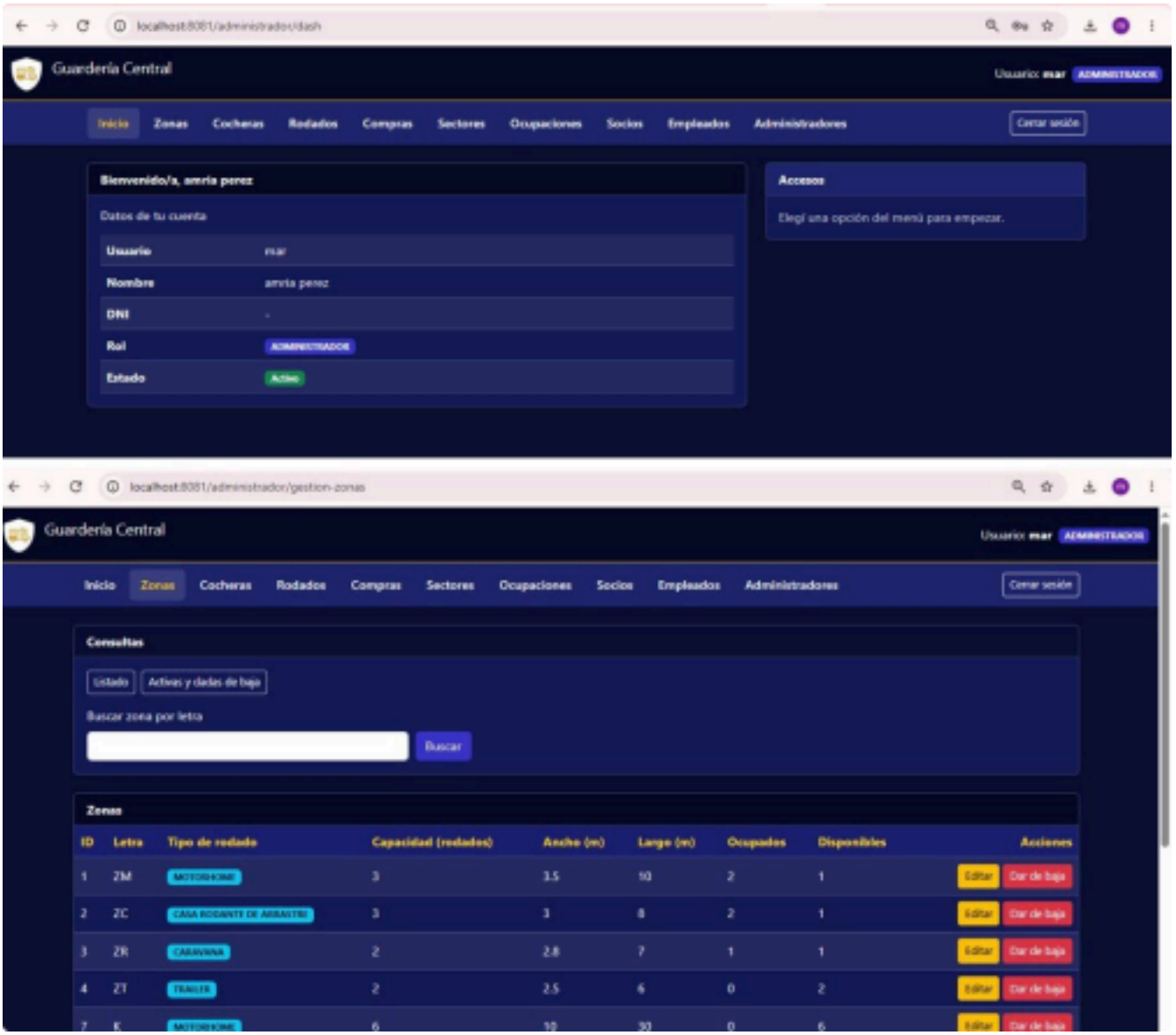
**Sectores:** código del empleado, letra de zona y cantidad de rodados a cargo. Se pueden buscar por código de empleado.

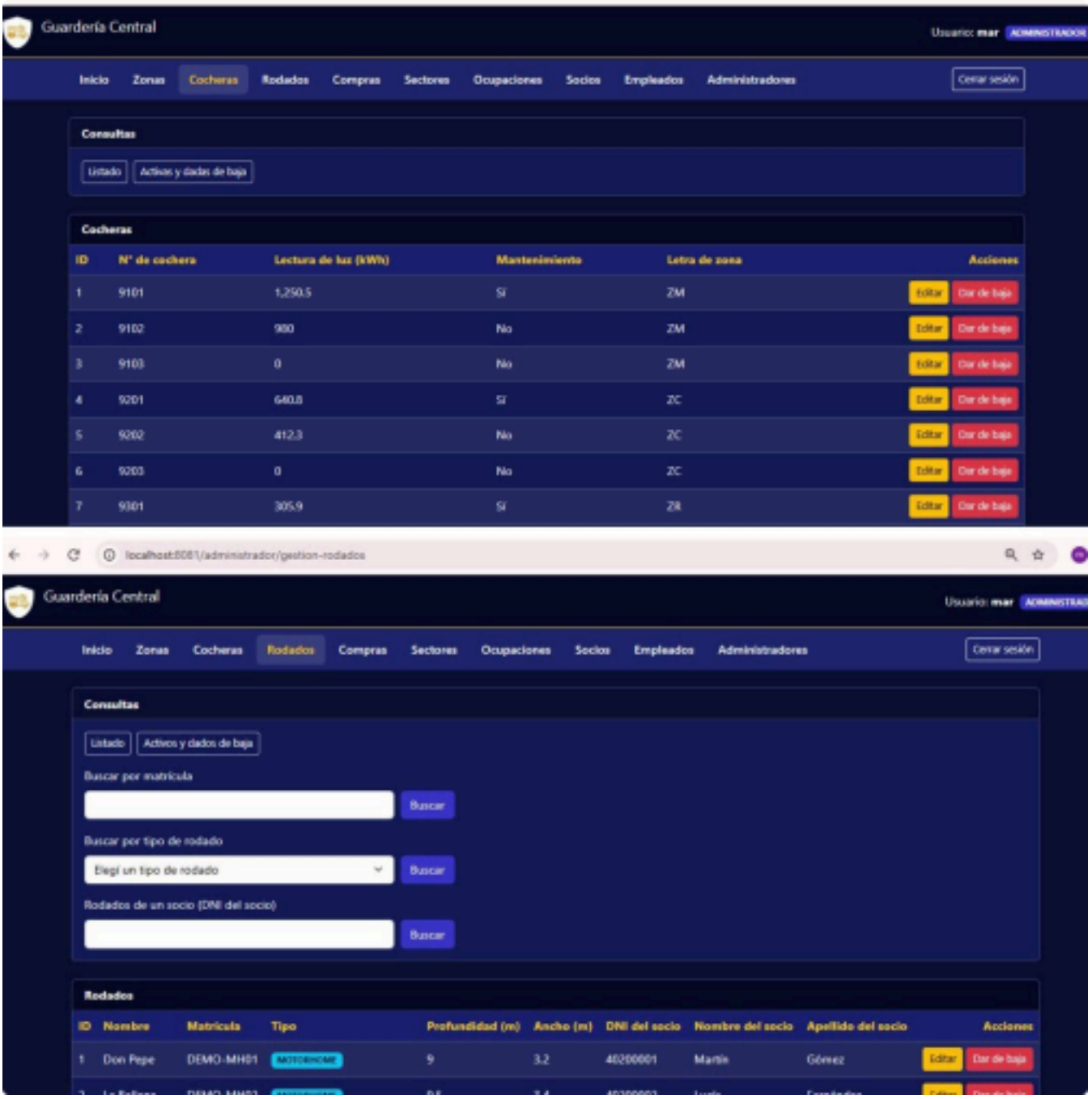
**Ocupaciones:** matrícula del rodado, número de cochera y fecha de asignación. Se pueden consultar por matrícula y ver el historial.

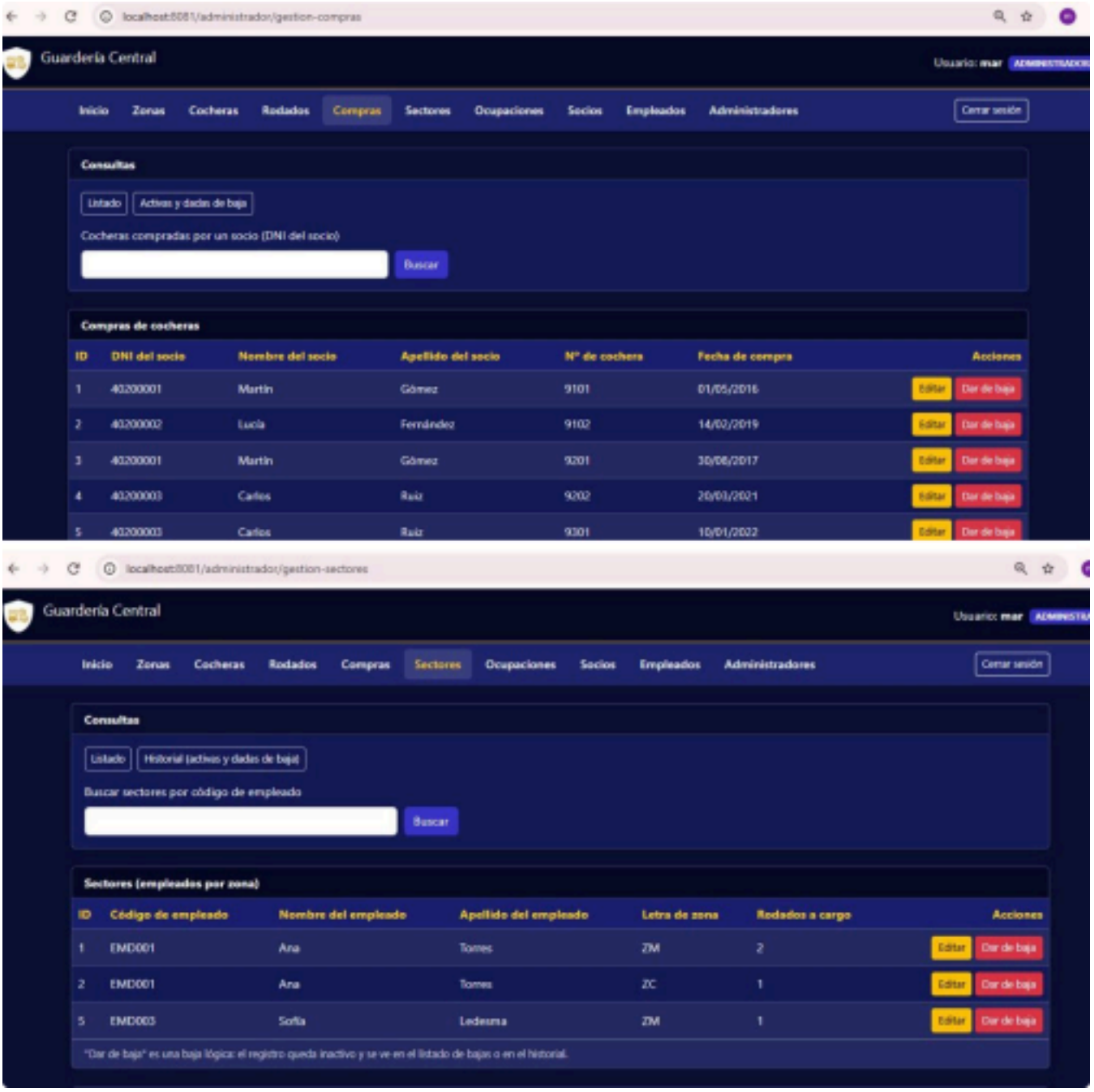
**Socios, Empleados yAdministradores:** el formulario se abre con **Registrar nuevo …**. En Socios, los botones **Rodados** y **Cocheras** de cada fila muestran lo que tiene ese socio. En Empleados se puede encargar un rodado por su matrícula y ver los sectores y encargos de cada empleado. En Administradores, la lista no incluye las cuentas SYSADMIN. Los botones **Ver bajas** y **Ver activos** alternan la vista.

**Orden recomendado para cargar datos:** zona, cocheras, socio, rodado, compra, ocupación, empleado y sector. Para dar de baja, se hace en el orden inverso.

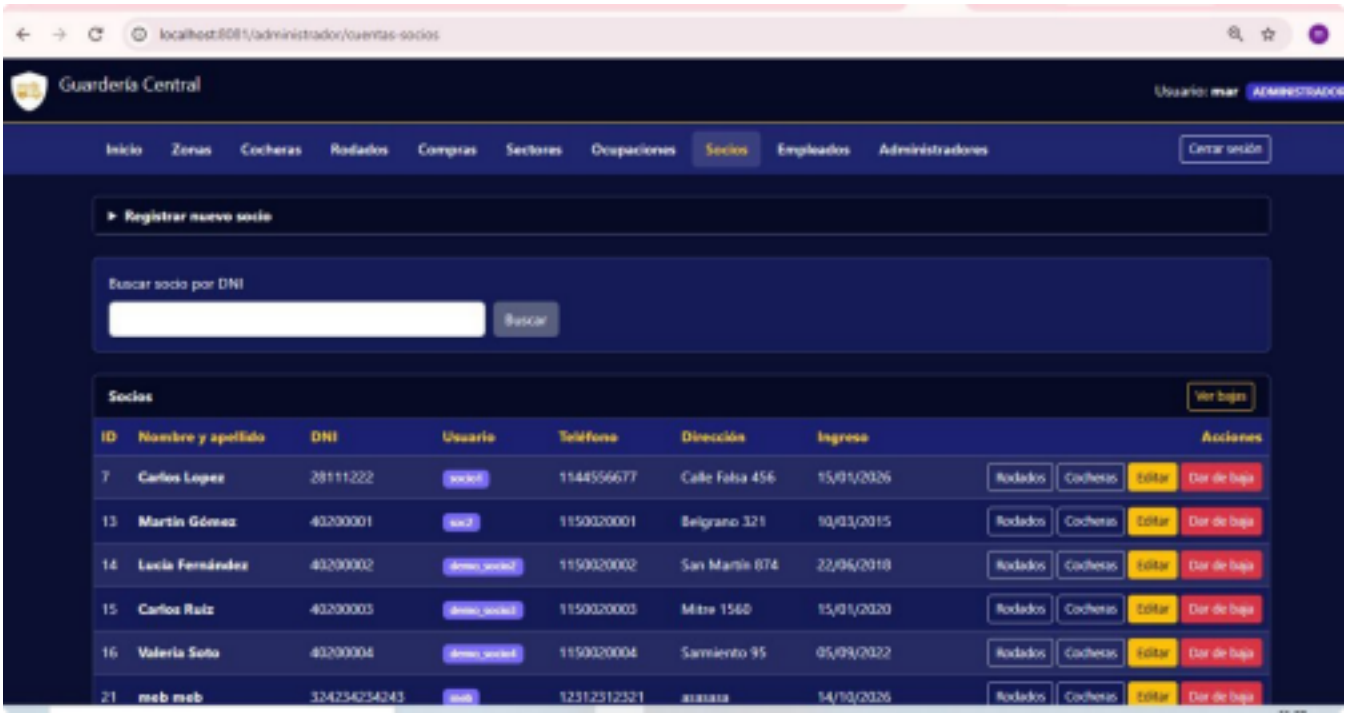
**Capturas del Administrador:**


Guardería Central
Usuario: mar
ADMINISTRADOR
Inicio
Zonas
Cocheras
Rodados
Compras
Sectores
Ocupaciones
Socios
Empleados
Administradores
Cerrar sesión
Bienvenido/a, amria perez
Datos de tu cuenta
Usuario
mar
Nombre
amria perez
DNI
Rol
ADMINISTRADOR
Estado
Activo
Accesos
Elegí una opción del menú para empezar.
Guardería Central
Usuario: mar
ADMINISTRADOR
Inicio
Zonas
Cocheras
Rodados
Compras
Sectores
Ocupaciones
Socios
Empleados
Administradores
Cerrar sesión
Consultas
Listado
Activas y dadas de baja
Buscar zona por letra
Buscar
Zonas
ID
Letra
Tipo de rodado
Capacidad (rodados)
Ancho (m)
Largo (m)
Ocupados
Disponibles
Acciones
1
ZM
MOTORHOME
3
3.5
10
2
1
Editar
Dar de baja
2
ZC
CASA RODANTE DE ARRASTRE
3
3
8
2
1
Editar
Dar de baja
3
ZR
CARAVANA
2
2.8
7
1
1
Editar
Dar de baja
4
ZT
TRAILER
2
2.5
6
0
2
Editar
Dar de baja

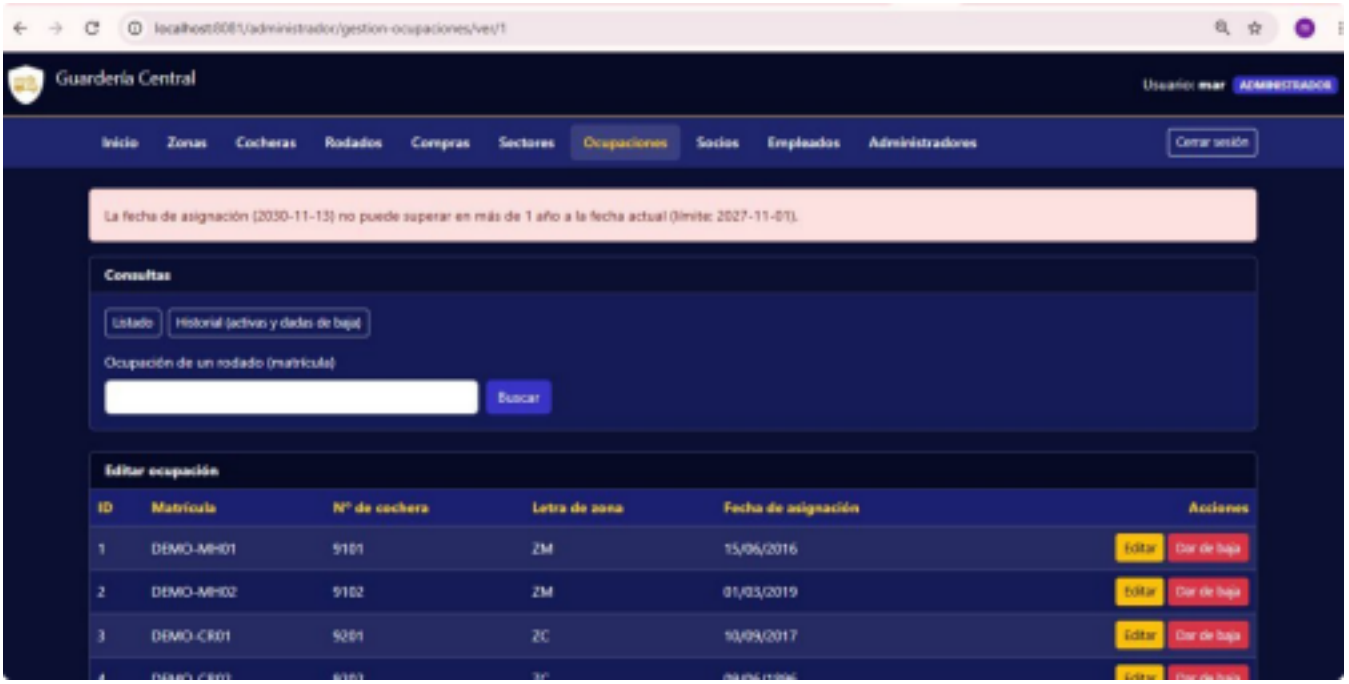

Guardería Central
Usuario: mar
Inicio
Zonas
Cocheras
Rodados
Compras
Sectores
Ocupaciones
Socios
Empleados
Administradores
Cerrar sesión
Consultas
Listado
Activos y dados de baja
Cocheras
ID
N° de cochera
Lectura de luz (kWh)
Mantenimiento
Letra de zona
Acciones
1 9101 1,250.5 Si ZM Editar Dar de baja
2 9102 980 No ZM Editar Dar de baja
3 9103 0 No ZM Editar Dar de baja
4 9201 640.8 Si ZC Editar Dar de baja
5 9202 412.3 No ZC Editar Dar de baja
6 9203 0 No ZC Editar Dar de baja
7 9301 305.9 Si ZR Editar Dar de baja
localhost:8081/administrador/gestion-rodados
Guardería Central
Usuario: mar
Inicio
Zonas
Cocheras
Rodados
Compras
Sectores
Ocupaciones
Socios
Empleados
Administradores
Cerrar sesión
Consultas
Listado
Activos y dados de baja
Buscar por matrícula
Buscar
Buscar por tipo de rodado
Elegí un tipo de rodado
Buscar
Rodados de un socio (DNI del socio)
Buscar
Rodados
ID
Nombre
Matrícula
Tipo
Profundidad (m)
Ancho (m)
DNI del socio
Nombre del socio
Apellido del socio
Acciones
1 Don Pepe DEMO-MH01 9 3.2 40200001 Martín Gómez Editar Dar de baja


localhost:8081/administrador/gestion-compras
Guardería Central
Usuario: mar
Inicio
Zonas
Cocheras
Rodados
Compras
Sectores
Ocupaciones
Socios
Empleados
Administradores
Cerrar sesión
Consultas
Listado
Activas y dadas de baja
Cocheras compradas por un socio (DNI del socio)
Buscar
Compras de cocheras
ID
DNI del socio
Nombre del socio
Apellido del socio
N° de cochera
Fecha de compra
Acciones
1 40200001 Martín Gómez 9101 01/05/2016 Editar Dar de baja
2 40200002 Lucía Fernández 9102 14/02/2019 Editar Dar de baja
3 40200001 Martín Gómez 9201 30/06/2017 Editar Dar de baja
4 40200003 Carlos Ruiz 9202 20/01/2021 Editar Dar de baja
5 40200003 Carlos Ruiz 9301 10/01/2022 Editar Dar de baja


localhost:8081/administrador/gestion-sectores
Guardería Central
Usuario: mar
Inicio
Zonas
Cocheras
Rodados
Compras
Sectores
Ocupaciones
Socios
Empleados
Administradores
Cerrar sesión
Consultas
Listado
Historial (activas y dadas de baja)
Buscar sectores por código de empleado
Buscar
Sectores (empleados por zona)
ID
Código de empleado
Nombre del empleado
Apellido del empleado
Letra de zona
Rodados a cargo
Acciones
1 EMD001 Ana Torres ZM 2 Editar Dar de baja
2 EMD001 Ana Torres ZC 1 Editar Dar de baja
5 EMD003 Sofía Ledesma ZM 1 Editar Dar de baja
"Dar de baja" es una baja lógica: el registro queda inactivo y se ve en el listado de bajas o en el historial.




**Ejemplo de mensaje de validación (fecha de asignación con más de 1 año hacia adelante):**



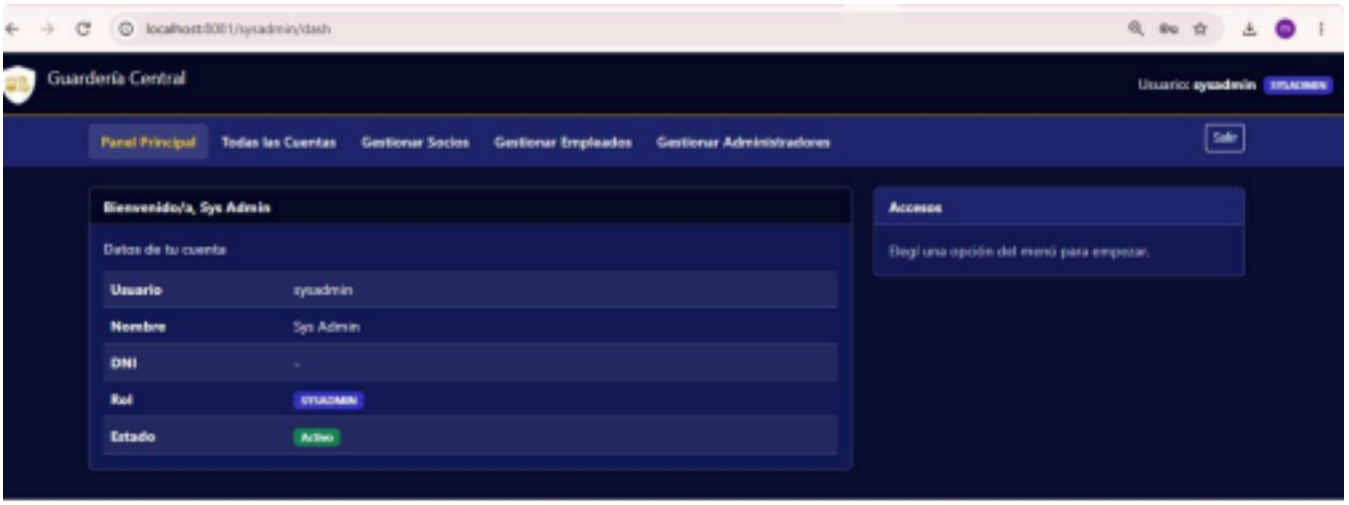
## Manual del SYSADMIN

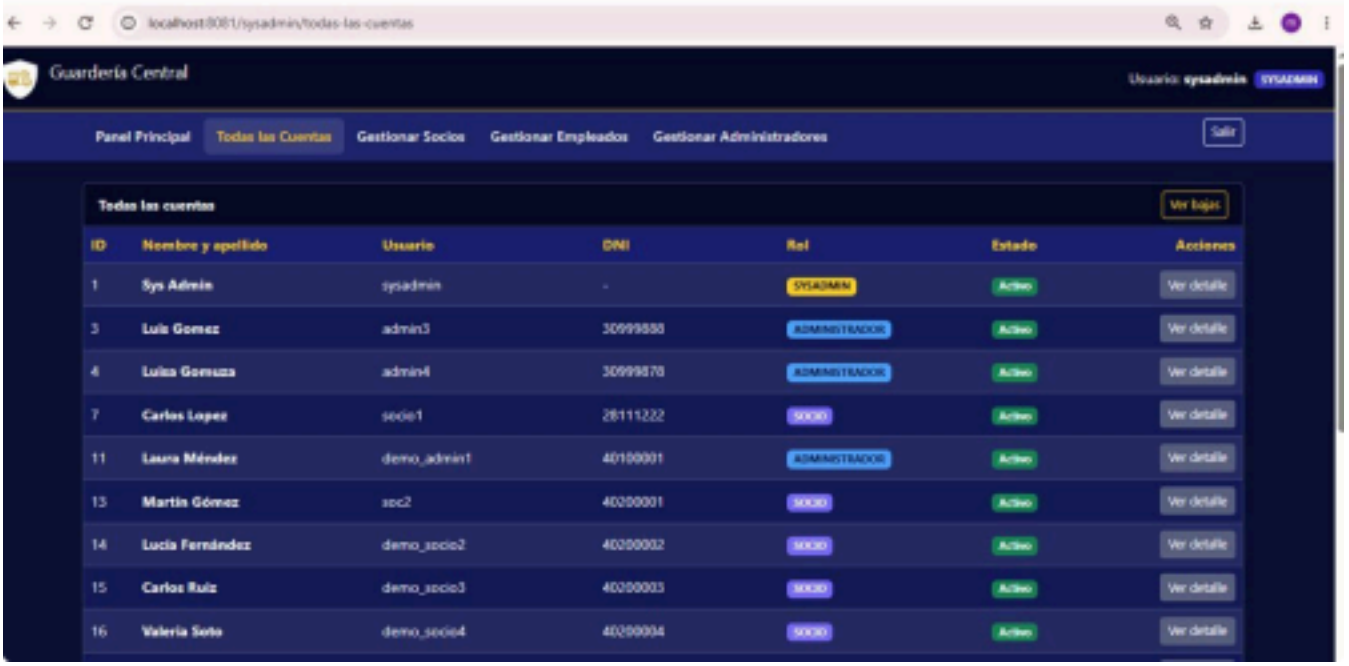
**Menú:** Panel Principal, Todas las Cuentas, Gestionar Socios, Gestionar Empleados, GestionarAdministradores.

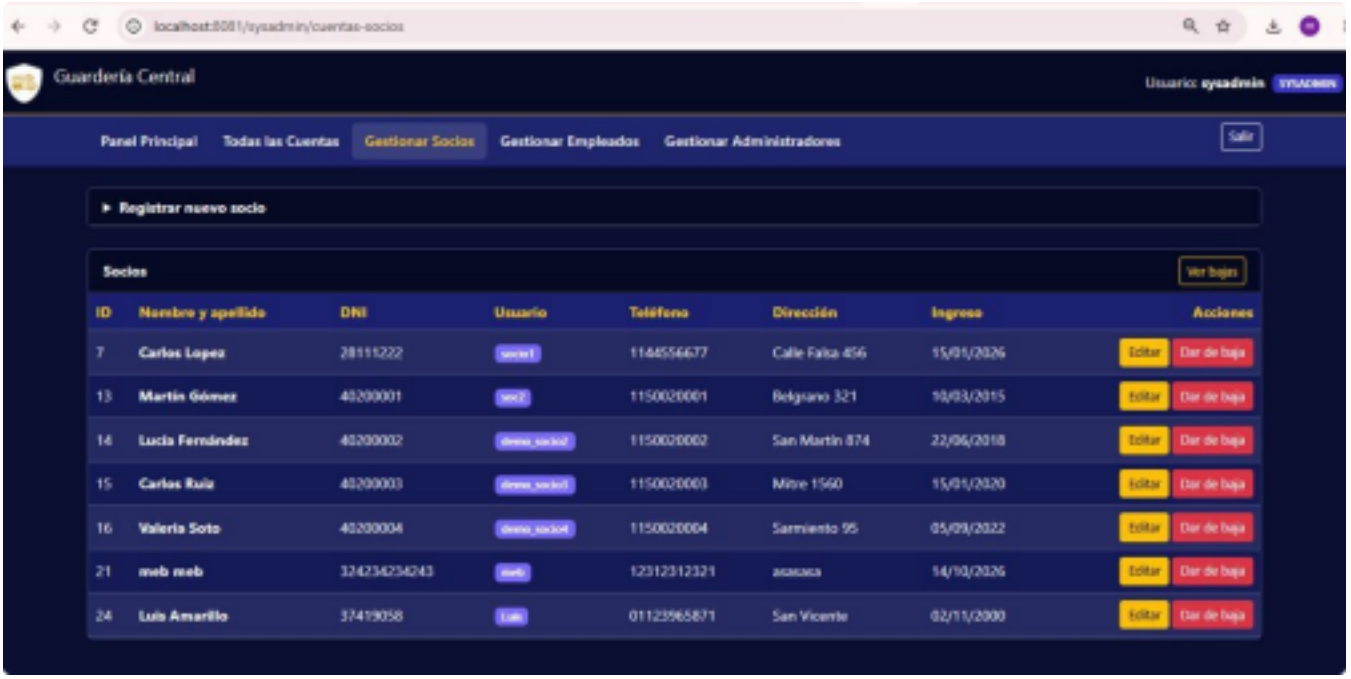
**Todas las Cuentas:** lista todas las cuentas del sistema (socios, empleados, administradores y SYSADMIN). Es de solo lectura: el botón **Ver detalle** muestra la ficha de cada una, y **Ver bajas** muestra también las dadas de baja.

**Gestionar Socios, Empleados yAdministradores:** igual que en el manual del Administrador. Se pueden registrar, editar y dar de baja cuentas.

## Capturas del SYSADMIN:






El SYSADMIN se crea solo al iniciar el backend, con el usuario y la clave que figuran en entorno/config.properties . El SYSADMIN no puede dar de baja su propia cuenta, para que el sistema no se quede sin SYSADMIN.

## 8. Decisiones de implementación

**Tecnologías:** Java 21, Spring Boot 4.1.1 con Maven y empaquetado WAR, Spring Data JPA con Hibernate, MySQL y Lombok. En el frontend, JSP con JSTL. Desarrollado con IntelliJ IDEA.

**Frameworks utilizados:** Spring Boot, Spring MVC, Spring Data JPA, Hibernate y Spring Security, que cubren y superan los dos frameworks que pide la consigna. **Dos aplicaciones separadas:** el backend (puerto 8080) no conoce las vistas y el frontend (puerto 8081) no accede a la base de datos. Así cada parte se puede probar y desplegar por separado, y el front se limita a consumir la API.

**Persistencia:** MySQL con JPA. Las tablas las genera Hibernate al iniciar ( ddl auto=update ), por lo que alcanza con crear la base vacía.

**Herencia JOINED:** se eligió para que los datos comunes de todas las cuentas (nombre, DNI, usuario, clave, rol) estén en una sola tabla y cada rol agregue solo lo propio. **Autenticación por sesión HTTP:** el backend devuelve una cookie de sesión ( GUARDERIA_BACK ) con vencimiento a los 30 minutos de inactividad. Las claves se guardan cifradas con BCrypt y nunca se devuelven en las respuestas. **Autorización centralizada:** todas las reglas de acceso por rol están en

SecurityConfig del backend (la primera regla que coincide es la que se aplica). Para saber qué puede hacer cada rol hay un solo lugar para mirar, y agregar un rol no obliga a tocar los controladores.

**Borrado lógico:** ningún registro se elimina físicamente. Cada entidad tiene el campo activo y se filtra con @SQLRestriction . Los historiales de bajas usan consultas nativas, porque esa restricción también filtra las consultas JPQL. Si se vuelve a dar de alta una cuenta dada de baja, se reactiva en lugar de duplicarse.

**Reglas de negocio en el service:** un registro no se puede dar de baja mientras otro activo dependa de él (un socio con rodados o compras, una cochera ocupada o comprada, una zona con cocheras o sectores). Cuando corresponde, la baja libera lo asociado: por ejemplo, dar de baja un empleado libera sus rodados a cargo y sus sectores.

**Validación de datos:** Bean Validation en los Request y Update , con expresiones regulares que solo admiten caracteres válidos, de modo que no entren scripts ni marcado.

**Manejo de errores:** excepciones de negocio propias (la mayoría hereda de BusinessException ) y un GlobalExceptionHandler que las traduce a JSON. El frontend muestra el mensaje del backend al usuario.

**Serialización con Jackson:** el backend usa Jackson (incluido en Spring Boot). Los DTO Request y Update se deserializan desde el JSON del cuerpo de la petición y los Response se serializan a JSON en la respuesta. En el frontend,

BackendClientService (con RestTemplate ) recibe ese JSON y Jackson lo convierte a Map / List<Map> ; al enviar datos, arma un Map que se serializa a JSON. **Registro de eventos (logging):** se usa SLF4J, con Logback incluido en Spring Boot, a través de @Slf4j de Lombok. Los controladores registran cada pedido recibido y los services registran las operaciones importantes y los intentos rechazados (por ejemplo, un rol no permitido). SLF4J es una fachada: el código escribe contra SLF4J y la implementación de logging se puede cambiar sin tocar las clases.

**Configuración sensible fuera del repositorio:** la clave de la base y las credenciales del SYSADMIN inicial están en entorno/config.properties , que no se sube a Git. Al iniciar, el backend crea el SYSADMIN si todavía no existe.

**Frontend sin JavaScript y sin DTO:** por consigna, las vistas usan solo formularios y JSTL, y los datos viajan como Map . Los controllers comparten un contrato de modelo ( lista , registro , puedeCrear , puedeEditar , puedeEliminar , modoBajas ) que permite reutilizar la misma vista para alta, edición e historial.

**Documentación:** Javadoc en español en las clases y README en cada repositorio con los pasos para configurar y ejecutar.

## 9. Uso de los principios de POO

**Encapsulamiento:** todos los atributos de las entidades son privados y se accede a ellos mediante getters y setters (generados con Lombok). Las respuestas de la API usan clases Response que exponen solo lo necesario: la clave nunca sale del backend. **Abstracción:** Usuario es una clase abstracta que representa lo común de cualquier cuenta y no se puede instanciar sola. Además, servicios y repositorios se definen como interfaces: las capas superiores conocen el contrato (qué se puede hacer) y no cómo se implementa.

**Herencia:** Socio , Empleado y Administrador heredan de Usuario y reutilizan sus atributos (nombre, apellido, DNI, nombre de usuario, clave, rol, activo), agregando solo lo propio de cada uno. La mayoría de las excepciones de negocio heredan de BusinessException , que fija el mensaje y el código HTTP.

**Polimorfismo:** una lista de Usuario puede contener socios, empleados y administradores a la vez, y se los trata de manera uniforme (por ejemplo, la vista "Todas las cuentas" del SYSADMIN). Los servicios también se usan a través de sus interfaces, de modo que el controlador funciona igual con cualquier implementación. GlobalExceptionHandler atiende a la familia de excepciones de negocio con un mismo tratamiento.

**Asociación y composición entre objetos:** un Vehiculo pertenece a un Socio y puede estar a cargo de un Empleado ; un Garage pertenece a una Zona ; una PropiedadGarage y una AsignacionVehiculoGarage vinculan cocheras con socios y rodados. El modelo refleja el dominio con relaciones entre objetos y no con identificadores sueltos.

## 10. Aplicación de los principios S.O.L.I.D.

**S (Single Responsibility):** cada clase tiene una sola responsabilidad. El controlador solo recibe y responde HTTP, el service aplica las reglas de negocio, el repositorio accede a datos, GlobalExceptionHandler traduce errores y SecurityConfig decide los permisos. En el front, BackendClientService es la única clase que habla con el backend.

**O (Open/Closed):** el sistema se extiende sin modificar lo existente. Para agregar un rol nuevo sin atributos propios se suma un valor al enum Rol y sus reglas en SecurityConfig ; el formulario de alta toma la lista de roles disponibles del backend, y los services, repositorios y controladores de cuentas no cambian. Las excepciones nuevas se agregan heredando de BusinessException , sin tocar el manejador global. **L (Liskov Substitution):** Socio , Empleado y Administrador pueden usarse donde se espera un Usuario sin alterar el comportamiento (por ejemplo, al listar todas las cuentas). Lo mismo ocurre con cada implementación de service respecto de su interfaz.

**I (Interface Segregation):** hay una interfaz de service por entidad ( SocioService, ZonaService , GarageService , etc.), cada una con solo las operaciones de su dominio, en lugar de una interfaz única y enorme. Ningún controlador depende de métodos que no usa.

**D (Dependency Inversion):** los controladores dependen de las interfaces de service y no de sus implementaciones, y los services dependen de las interfaces de repositorio. Spring inyecta las dependencias por constructor, lo que desacopla las capas y permite reemplazar una implementación sin tocar quien la usa.

## 11. Aclaraciones relevantes

**Orden para ejecutar:** primero la base de datos MySQL (con la base guarderia_db creada), después el backend y por último el frontend. El backend necesita el archivo entorno/config.properties , que no está en el repositorio; el README explica su contenido.

**Orden para cargar datos:** para cargar datos conviene respetar el orden de dependencias: zona, cocheras, socio, rodado, compra, ocupación, empleado y sector. Un registro no se puede dar de baja mientras otro activo dependa de él.

**Términos de la consigna:** las pantallas y mensajes usan "cochera" y "rodado"; en el código y las rutas los nombres son garage y vehículo .

**Acceso desde otras PC:** para ver la web desde otro equipo, el frontend se abre con la IP de la PC donde corre ( http://<IP>:8081/login ), se permite el puerto 8081 en el firewall y el origen del frontend se informa al backend en app.cors.allowed-origin .

**Requisito de red del frontend:** los estilos de Bootstrap se cargan desde un CDN, por lo que la PC que muestra la web necesita acceso a internet.

**Diagramas y documentación complementaria:** el diagrama de casos de uso, el diagrama entidad-relación y los diagramas de clases (UML) están en la carpeta documentación del repositorio del backend.